

SENAI - CRTI

Apostila

Segurança na Integração: Login, Token e Rotas Protegidas

Projeto guiado: DummyJSON — autenticação com JWT, interceptor e route guard, passo a passo para o aluno replicar sozinho

Interação com APIs — Sprint 3

Sumário

- 1. Introdução e pré-requisitos
- 2. Conceitos básicos
- 3. Passo a passo: criando o projeto pelo terminal
- 4. Código-fonte completo, arquivo por arquivo
- 5. Executando e testando a aplicação
- 6. Testes automatizados
- 7. Build de produção

1. Introdução e pré-requisitos

Nesta apostila você vai construir, do zero, a aplicação Angular DummyJSON: uma tela de login que autentica contra uma API real, guarda o token de sessão, protege uma rota interna com um Route Guard e injeta o token automaticamente em toda requisição HTTP através de um Interceptor. Todas as operações conversam com a API pública DummyJSON, que devolve um JWT real de autenticação — ideal para treinar o fluxo completo de login antes de ligar a uma API de produção.

Siga os capítulos na ordem. Cada comando de terminal e cada arquivo de código aparecem completos, prontos para copiar.

Pré-requisitos

- Node.js instalado (versão LTS mais recente).
- Angular CLI instalado globalmente.
- Um editor de código (ex.: VS Code).

Para instalar o Angular CLI globalmente, caso ainda não tenha:

```
$ npm install -g @angular/cli
```

Para conferir se tudo está pronto, rode os três comandos abaixo. Eles devem responder com um número de versão, sem erro:

```
$ node --version
$ npm --version
$ ng version
```

2. Conceitos básicos

Antes de programar, é importante entender o papel de cada peça que vamos usar.

HttpClient

É o serviço oficial do Angular para fazer requisições HTTP. Nesta apostila usamos `post()` para enviar o login e `get()` para buscar os dados do usuário autenticado.

Autenticação × Autorização

Autenticação é o processo de confirmar quem o usuário é (login com usuário e senha, devolvendo um token). Autorização é decidir o que esse usuário autenticado pode acessar. O AuthService cuida da autenticação; o Route Guard cuida da autorização de rotas.

Token JWT e cabeçalho Bearer

Um JWT (JSON Web Token) é uma string assinada que a API devolve depois de um login válido. A partir daí, toda requisição a uma rota protegida precisa enviar esse token no cabeçalho HTTP Authorization, no formato

Authorization: Bearer <token>. É assim que a API sabe quem está fazendo a requisição sem precisar de cookies de sessão.

AuthService

Uma classe decorada com `@Service()` — o decorator padrão do Angular 22 para services singleton — responsável por três coisas: fazer o login (POST), guardar a sessão (token + dados do usuário) e expor essa sessão para o resto da aplicação através de signals. É também o AuthService quem sabe devolver o token atual para o interceptor.

HTTP Interceptor

Uma função (`HttpInterceptorFn`) que intercepta toda requisição HTTP feita pela aplicação antes dela sair, e toda resposta antes dela chegar ao código que pediu. Usamos o interceptor para duas coisas: adicionar o cabeçalho Authorization automaticamente (assim nenhum service precisa fazer isso na mão) e reagir a uma resposta 401 Unauthorized deslogando o usuário e mandando ele de volta para o login.

Route Guard

Uma função (`CanActivateFn`) que o Angular Router consulta antes de ativar uma rota. Se o guard devolver `true`, a navegação continua; se devolver uma URL (via `router.createUrlTree(...)`), o usuário é redirecionado para lá em vez de acessar a rota original. Usamos o `authGuard` para impedir que a rota `/perfil` seja aberta por alguém que não fez login.

Por que sessionStorage?

`sessionStorage` guarda dados apenas enquanto a aba do navegador estiver aberta — ao fechar a aba, a sessão some. `localStorage`, por outro lado, persiste indefinidamente, mesmo depois de fechar o navegador. Para um token de autenticação, `sessionStorage` é geralmente a escolha mais segura: reduz o tempo em que um token roubado continuaria válido no armazenamento do navegador.

Model (Interfaces)

Interfaces TypeScript que descrevem o formato dos dados: `Login` (o que enviamos), `UsuarioAutenticado` (o que a API devolve no login, incluindo os tokens) e `PerfilCompleto` (o que a API devolve na rota autenticada `/auth/me`). Isso dá autocomplete e evita erros de digitação nos nomes dos campos.

Signals

Signals são variáveis reativas do Angular. Quando o valor de um signal muda (ex.: `signal.set(novoValor)`), a tela é atualizada automaticamente, sem precisar de comandos manuais de renderização. O AuthService expõe a sessão atual como um signal computado, para que qualquer componente saiba, em tempo real, se há um usuário autenticado.

Observable e subscribe

O `HttpClient` não devolve o dado na hora — ele devolve um `Observable`, que representa uma resposta que vai chegar no futuro. Para receber esse valor quando ele chegar (ou tratar um erro), usamos `.subscribe({ next, error })`.

Zoneless (Angular 21+)

Desde a versão 21, o Angular não depende mais da biblioteca Zone.js para saber quando atualizar a tela. Em vez disso, a atualização acontece diretamente quando um Signal muda de valor. Isso deixa a aplicação mais leve e rápida, sem exigir nenhuma configuração extra — já vem assim por padrão em projetos novos.

Testando código que usa HttpClient

Quando um teste precisa simular uma chamada HTTP em vez de acessar a internet de verdade, trocamos `provideHttpClient()` por `provideHttpClient() + provideHttpClientTesting()`, e usamos um `HttpTestingController` para interceptar a requisição e devolver uma resposta falsa.

Resumo: ação da tela → mecanismo → onde acontece

- Preencher o formulário e clicar em "Entrar" → POST → `authService.login(credenciais)`
- Login com sucesso → guarda token e dados do usuário → `authService.salvarSessao(usuario)`
- Toda requisição HTTP feita depois do login → injeta o cabeçalho Authorization → `authInterceptor`
- Acessar /perfil → confere se há sessão ativa, senão redireciona ao /login → `authGuard`
- A API responde 401 em qualquer chamada → desloga e redireciona ao /login → `authInterceptor`
- Clicar em "Sair" → limpa a sessão e redireciona ao /login → `authService.logout()`

3. Passo a passo: criando o projeto pelo terminal

Abra o terminal na pasta onde deseja criar o projeto e siga os comandos na ordem.

3.1 Criar o projeto Angular

```
$ ng new DummyJSON --style=css --routing=true --ssr=false --defaults
```

O que cada opção faz: `--style=css` usa CSS puro para os estilos; `--routing=true` porque, diferente dos projetos anteriores, esta aplicação tem mais de uma tela (login e área protegida) e precisa de rotas; `--ssr=false` porque a aplicação roda só no navegador (client-side), sem Server-Side Rendering; `--defaults` aceita as demais respostas padrão automaticamente, sem perguntas interativas (o Angular 22 já cria o projeto `zoneless` por padrão).

3.2 Entrar na pasta do projeto

```
$ cd DummyJSON
```

3.3 Gerar o model (interface) de Usuário

```
$ ng generate interface core/models/usuario
```

Esse comando cria o arquivo `src/app/core/models/usuario.ts`, com uma interface `Usuario` vazia. Vamos substituir esse conteúdo por três interfaces — `Login`, `UsuarioAutenticado` e `PerfilCompleto` — no capítulo 4.

3.4 Gerar o AuthService

```
$ ng generate service core/services/auth
```

Esse comando cria dois arquivos: `src/app/core/services/auth.ts` (o service, já decorado com `@Service()`) e `src/app/core/services/auth.spec.ts` (o arquivo de teste). Por padrão, o Angular CLI nomeia a classe gerada apenas como `Auth` — renomeie a classe do service para `AuthService` (o nome do arquivo continua `auth.ts`, seguindo a convenção do Angular 22 de não usar mais sufixos de tipo nos nomes de arquivo).

3.5 Gerar o Interceptor de autenticação

```
$ ng generate interceptor core/interceptors/auth --skip-tests
```

Cria `src/app/core/interceptors/auth-interceptor.ts`, já como uma função `HttpInterceptorFn` chamada `authInterceptor` — o Angular CLI 22 deriva esse nome automaticamente do caminho informado. Usamos `--skip-tests` porque o teste desse interceptor específico não faz parte do escopo desta apostila.

3.6 Gerar o Route Guard de autenticação

```
$ ng generate guard core/auth --skip-tests
```

O CLI pergunta qual tipo de guard você quer criar — escolha `CanActivate` (opção padrão, usada para bloquear o acesso a uma rota inteira). Isso cria `src/app/core/auth-guard.ts`, já como uma função `CanActivateFn` chamada `authGuard`.

3.7 Gerar as telas de Login e Perfil

```
$ ng generate component features/login --skip-tests
```

```
$ ng generate component features/perfil --skip-tests
```

Cada comando cria uma pasta com quatro arquivos: o componente (.ts), o template (.html), o estilo (.css) — o Angular CLI 22 já não gera mais sufixo "component" em nenhum desses nomes. Usamos --skip-tests porque os testes destes dois componentes não fazem parte do escopo desta apostila.

3.8 Conectar HttpClient, Interceptor, Router e Guard

Os comandos acima criaram os arquivos, mas ainda não os conectaram entre si. No capítulo 4, vamos editar manualmente src/app/app.config.ts (para registrar o HttpClient com o interceptor) e src/app/app.routes.ts (para registrar as rotas /login e /perfil, esta última protegida pelo guard) — não existe comando de terminal para essa etapa, ela é sempre feita no código.

3.9 Estrutura final de pastas

Depois de todos os comandos, e com os arquivos preenchidos (próximo capítulo), a estrutura deve ficar assim:

```
src/  
  main.ts  
  app/  
    app.config.ts  
    app.routes.ts  
    app.ts  
    app.html  
    app.css  
    app.spec.ts  
  core/  
    auth-guard.ts  
    interceptors/  
      auth-interceptor.ts  
    models/  
      usuario.ts  
    services/  
      auth.ts  
      auth.spec.ts  
  features/  
    login/  
      login.ts  
      login.html  
      login.css  
    perfil/  
      perfil.ts  
      perfil.html  
      perfil.css
```

4. Código-fonte completo, arquivo por arquivo

A seguir está o conteúdo final de cada arquivo. Abra cada um no editor e substitua o conteúdo pelo código abaixo.

4.1 src/main.ts

Ponto de entrada da aplicação. Inicializa (bootstrap) o componente principal App usando as configurações definidas em appConfig. Este arquivo é gerado pronto pelo Angular CLI — não precisa alterá-lo.

Arquivo: src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

4.2 src/app/app.config.ts

Configura os providers globais da aplicação: provideRouter(routes) habilita a navegação entre telas; provideHttpClient(withInterceptors([authInterceptor])) habilita o HttpClient já registrando o interceptor de autenticação — a partir daqui, toda requisição feita pela aplicação passa primeiro pelo authInterceptor.

Arquivo: src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';

import { routes } from './app.routes';
import { authInterceptor } from './core/interceptors/auth-interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
};
```

4.3 src/app/app.routes.ts

Define duas rotas: /login (pública) e /perfil (protegida pelo authGuard no campo canActivate). A rota raiz redireciona para /login, e qualquer URL desconhecida (**) também. Ambos os componentes são carregados de forma lazy (loadComponent), ou seja, o código de cada tela só é baixado quando o usuário navega até ela.

Arquivo: src/app/app.routes.ts

```
import { Routes } from '@angular/router';

import { authGuard } from '../core/auth-guard';

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'login' },
  {
    path: 'login',
    loadComponent: () => import('../features/login/login').then((m) => m.Login),
  },
  {
    path: 'perfil',
    canActivate: [authGuard],
    loadComponent: () => import('../features/perfil/perfil').then((m) => m.Perfil),
  },
  { path: '**', redirectTo: '' },
];
```

4.4 src/app/core/models/usuario.ts

Login descreve o payload enviado ao fazer login. UsuarioAutenticado descreve o formato real que a DummyJSON devolve em POST /auth/login — incluindo accessToken e refreshToken, os dois JWTs da sessão. PerfilCompleto descreve o formato (simplificado, só os campos que a tela usa) devolvido pela rota protegida GET /auth/me.

Arquivo: src/app/core/models/usuario.ts

```
export interface Login {
  username: string;
  password: string;
}

export interface UsuarioAutenticado {
  id: number;
  username: string;
  email: string;
  firstName: string;
  lastName: string;
  gender: string;
  image: string;
  accessToken: string;
  refreshToken: string;
}

export interface PerfilCompleto {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  role: string;
}
```

```
    image: string;
  }
```

4.5 src/app/core/services/auth.ts

O AuthService concentra toda a lógica de autenticação. O signal privado `usuario` começa com o valor salvo em `sessionStorage` (se existir), o que mantém o usuário logado mesmo depois de recarregar a página. `usuarioAtual` e `estaAutenticado` são computed signals de leitura, usados pelos componentes e pelo guard. `login()` faz o POST e devolve o Observable — quem chama decide quando assinar. `salvarSessao()` é chamada pelo componente de login após a resposta chegar, e é ela quem persiste a sessão em `sessionStorage`. `buscarPerfilCompleto()` faz uma chamada GET separada e autenticada, usada na tela de perfil para provar que o interceptor está funcionando. `obterToken()` é usada pelo interceptor para montar o cabeçalho `Authorization`.

Arquivo: src/app/core/services/auth.ts

```
import { Injectable, computed, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';

import { Login, PerfilCompleto, UsuarioAutenticado } from '../models/usuario';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);

  private readonly API_LOGIN = 'https://dummyjson.com/auth/login';
  private readonly API_PERFIL = 'https://dummyjson.com/auth/me';
  private readonly CHAVE_SESSAO = 'dummyjson-usuario';

  private usuario = signal<UsuarioAutenticado | null>(this.carregarSessaoSalva());

  usuarioAtual = computed(() => this.usuario());
  estaAutenticado = computed(() => this.usuario() !== null);

  login(credenciais: Login) {
    return this.http.post<UsuarioAutenticado>(this.API_LOGIN, credenciais);
  }

  buscarPerfilCompleto() {
    return this.http.get<PerfilCompleto>(this.API_PERFIL);
  }

  salvarSessao(usuario: UsuarioAutenticado) {
    this.usuario.set(usuario);
    sessionStorage.setItem(this.CHAVE_SESSAO, JSON.stringify(usuario));
  }

  logout() {
    this.usuario.set(null);
    sessionStorage.removeItem(this.CHAVE_SESSAO);
  }
}
```

```

obterToken(): string | null {
  return this.usuario()?.accessToken ?? null;
}

private carregarSessaoSalva(): UsuarioAutenticado | null {
  const dadosSalvos = sessionStorage.getItem(this.CHAVE_SESSAO);

  if (!dadosSalvos) {
    return null;
  }

  try {
    return JSON.parse(dadosSalvos) as UsuarioAutenticado;
  } catch {
    return null;
  }
}
}

```

4.6 src/app/core/interceptors/auth-interceptor.ts

Intercepta toda requisição feita pela aplicação. Se houver um token salvo, clona a requisição original adicionando o cabeçalho Authorization — a requisição em si nunca é modificada diretamente, porque objetos `HttpRequest` são imutáveis no Angular. Depois de repassar a requisição, o `catchError` observa a resposta: se o status for 401 (token inválido ou expirado), desloga o usuário e redireciona para `/login` antes de propagar o erro adiante.

Arquivo: src/app/core/interceptors/auth-interceptor.ts

```

import { inject } from '@angular/core';
import { HttpInterceptorFn } from '@angular/common/http';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';

import { AuthService } from '../services/auth';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  const token = authService.obterToken();

  const requisicaoAutenticada = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(requisicaoAutenticada).pipe(
    catchError((erro) => {
      if (erro.status === 401) {

```

```

        authService.logout();
        router.navigateByUrl('/login');
    }

    return throwError(() => erro);
  }},
);
};

```

4.7 src/app/core/auth-guard.ts

Consultado pelo Angular Router antes de ativar a rota /perfil. Se houver sessão ativa (estaAutenticado() true), libera a navegação devolvendo true. Caso contrário, devolve uma UrlTree apontando para /login — o Router entende isso como "redirecione para cá em vez disso".

Arquivo: src/app/core/auth-guard.ts

```

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

import { AuthService } from '../services/auth';

export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.estaAutenticado()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};

```

4.8 src/app/features/login/login.ts

Formulário reativo com dois campos obrigatórios (username e password). Ao enviar, se o formulário for inválido, mostra uma mensagem sem chamar a API. Se for válido, chama authService.login() — no sucesso, salva a sessão e navega para /perfil; no erro, distingue um 400 (credenciais erradas, mensagem específica) de qualquer outro problema (mensagem genérica).

Arquivo: src/app/features/login/login.ts

```

import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { HttpResponse } from '@angular/common/http';
import { Router } from '@angular/router';

import { AuthService } from '../../core/services/auth';

```

```

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css'],
})
export class Login {
  private authService = inject(AuthService);
  private router = inject(Router);

  carregando = signal(false);
  erro = signal('');

  formulario = new FormGroup({
    username: new FormControl('', {
      nullable: true,
      validators: [Validators.required],
    }),
    password: new FormControl('', {
      nullable: true,
      validators: [Validators.required],
    }),
  });

  entrar() {
    if (this.formulario.invalid) {
      this.formulario.markAllAsTouched();
      this.erro.set('Preencha usuário e senha.');
      return;
    }

    this.carregando.set(true);
    this.erro.set('');

    this.authService.login(this.formulario.getRawValue()).subscribe({
      next: (usuario) => {
        this.authService.salvarSessao(usuario);
        this.carregando.set(false);
        this.router.navigateByUrl('/perfil');
      },
      error: (erro: HttpErrorResponse) => {
        this.carregando.set(false);
        this.erro.set(
          erro.status === 400
            ? 'Usuário ou senha inválidos.'
            : 'Não foi possível fazer login. Tente novamente.',
        );
      },
    });
  }
}

```

4.9 src/app/features/login/login.html

Template do login: um cartão centralizado com uma dica visível dos dados de teste, os dois campos do formulário, a mensagem de erro (quando existir) e o botão de envio, desabilitado durante o carregando().

Arquivo: src/app/features/login/login.html

```
<div class="container">
  <form class="cartao" [formGroup]="formulario" (ngSubmit)="entrar()">
    <h1>DummyJSON Login</h1>
    <p class="dica">
      Usuário de teste: <strong>emilys</strong> · Senha: <strong>emilyspass</strong>
    </p>

    <label>
      Usuário
      <input type="text" formControlName="username" placeholder="username" />
    </label>

    <label>
      Senha
      <input type="password" formControlName="password" placeholder="password" />
    </label>

    @if (erro()) {
      <p class="erro">{{ erro() }}</p>
    }

    <button type="submit" [disabled]="carregando()">
      {{ carregando() ? 'Entrando...' : 'Entrar' }}
    </button>
  </form>
</div>
```

4.10 src/app/features/login/login.css

Estilos da tela de login: cartão branco centralizado na tela, caixa de dica em azul claro, campos empilhados e botão que muda de cor ao passar o mouse ou fica acinzentado quando desabilitado.

Arquivo: src/app/features/login/login.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

:host {
```

```
display: block;
background: #f5f7fb;
min-height: 100vh;
}

.container {
display: flex;
justify-content: center;
align-items: center;
min-height: 100vh;
padding: 20px;
}

.cartao {
display: flex;
flex-direction: column;
gap: 14px;
width: 100%;
max-width: 360px;
background: white;
padding: 30px;
border-radius: 12px;
box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
}

h1 {
text-align: center;
font-size: 22px;
color: #222;
}

.dica {
text-align: center;
font-size: 13px;
color: #666;
background: #f0f4ff;
padding: 10px;
border-radius: 8px;
}

label {
display: flex;
flex-direction: column;
gap: 6px;
font-size: 14px;
color: #444;
font-weight: bold;
}
```

```

input {
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 15px;
  font-weight: normal;
}

button {
  border: none;
  background: #2563eb;
  color: white;
  padding: 14px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
  font-size: 15px;
}

button:hover:not(:disabled) {
  background: #1d4ed8;
}

button:disabled {
  background: #93c5fd;
  cursor: not-allowed;
}

.erro {
  color: #dc2626;
  font-weight: bold;
  font-size: 14px;
}

```

4.11 src/app/features/perfil/perfil.ts

A tela protegida. `usuario` expõe a sessão salva no login (leitura direta do signal do `AuthService`). No construtor, chama `carregarPerfilCompleto()`, que faz uma nova chamada GET a `/auth/me` — propositalmente separada dos dados já salvos no login, para provar visualmente que o interceptor está injetando o token sozinho nessa nova requisição. `sair()` limpa a sessão e volta para `/login`.

Arquivo: `src/app/features/perfil/perfil.ts`

```

import { Component, inject, signal } from '@angular/core';
import { Router } from '@angular/router';

import { AuthService } from '../../../core/services/auth';
import { PerfilCompleto } from '../../../core/models/usuario';

```

```

@Component({
  selector: 'app-perfil',
  imports: [],
  templateUrl: './perfil.html',
  styleUrls: ['./perfil.css'],
})
export class Perfil {
  private authService = inject(AuthService);
  private router = inject(Router);

  usuario = this.authService.usuarioAtual;

  perfilCompleto = signal<PerfilCompleto | null>(null);
  carregando = signal(true);
  erro = signal('');

  constructor() {
    this.carregarPerfilCompleto();
  }

  carregarPerfilCompleto() {
    this.carregando.set(true);
    this.erro.set('');

    this.authService.buscarPerfilCompleto().subscribe({
      next: (perfil) => {
        this.perfilCompleto.set(perfil);
        this.carregando.set(false);
      },
      error: () => {
        this.erro.set('Não foi possível confirmar sua sessão com a API. Tente novamente.');
```

4.12 src/app/features/perfil/perfil.html

Template da área protegida: um cabeçalho com o botão Sair, os dados da sessão local salvos no login, e uma seção separada — com seu próprio estado de carregando/erro/sucesso — que mostra o resultado da chamada autenticada a /auth/me.

Arquivo: *src/app/features/perfil/perfil.html*

```
<div class="container">
```

```

<div class="cartao">
  <header class="topo">
    <h1>Área protegida</h1>
    <button type="button" class="btn-sair" (click)="sair()">Sair</button>
  </header>

  @if (usuario(); as sessao) {
    <section class="sessao">
      <img [src]="sessao.image" [alt]="sessao.username" />
      <div>
        <p><strong>Sessão local (salva no login):</strong></p>
        <p>{{ sessao.firstName }} {{ sessao.lastName }} ({{ sessao.username }})</p>
        <p>{{ sessao.email }}</p>
      </div>
    </section>
  }

  <hr />

  <h2>Chamada autenticada a /auth/me</h2>
  <p class="explicacao">
    Esta seção não usa os dados salvos no login – ela faz uma nova requisição HTTP,
    autenticada automaticamente pelo interceptor com o token guardado na sessão.
  </p>

  @if (carregando()) {
    <p>Confirmando sessão com a API...</p>
  }

  @if (erro()) {
    <section class="mensagem-erro">
      <p>{{ erro() }}</p>
      <button type="button" (click)="carregarPerfilCompleto()">Tentar novamente</button>
    </section>
  }

  @if (perfilCompleto(); as perfil) {
    <ul class="detalhes">
      <li><strong>Perfil (role):</strong> {{ perfil.role }}</li>
      <li>
        <strong>Nome completo:</strong>
        {{ perfil.firstName }} {{ perfil.lastName }}
      </li>
      <li><strong>E-mail:</strong> {{ perfil.email }}</li>
    </ul>
  }
</div>
</div>

```

4.13 src/app/features/perfil/perfil.css

Estilos da área protegida: cartão branco, cabeçalho com o botão Sair alinhado à direita, avatar circular, lista de detalhes do perfil e a caixa de erro com o botão de tentar novamente.

Arquivo: src/app/features/perfil/perfil.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

:host {
  display: block;
  background: #f5f7fb;
  min-height: 100vh;
}

.container {
  display: flex;
  justify-content: center;
  padding: 40px 20px;
}

.cartao {
  width: 100%;
  max-width: 520px;
  background: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
}

.topo {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}

h1 {
  font-size: 20px;
  color: #222;
}

h2 {
  font-size: 16px;
  color: #222;
  margin-bottom: 8px;
}

.btn-sair {
```

```
border: none;
background: #fee2e2;
color: #b91c1c;
padding: 8px 16px;
border-radius: 8px;
cursor: pointer;
font-weight: bold;
font-size: 13px;
}

.btn-sair:hover {
  background: #fecaca;
}

.sessao {
  display: flex;
  gap: 14px;
  align-items: center;
}

.sessao img {
  width: 56px;
  height: 56px;
  border-radius: 50%;
}

.sessao p {
  color: #444;
  font-size: 14px;
  margin: 2px 0;
}

hr {
  border: none;
  border-top: 1px solid #eee;
  margin: 20px 0;
}

.explicacao {
  color: #666;
  font-size: 13px;
  margin-bottom: 14px;
}

.detalhes {
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 6px;
```

```

}

.detalhes li {
  color: #444;
  font-size: 14px;
}

.mensagem-erro {
  display: flex;
  flex-direction: column;
  gap: 10px;
  align-items: flex-start;
}

.mensagem-erro p {
  color: #dc2626;
  font-weight: bold;
  font-size: 14px;
}

.mensagem-erro button {
  border: none;
  background: #2563eb;
  color: white;
  padding: 8px 16px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
  font-size: 13px;
}

```

4.14 src/app/app.ts e src/app/app.html

O componente raiz. O Angular CLI gera esses arquivos com um template de boas-vindas — substitua o conteúdo por uma versão mínima, já que quem decide o que aparece na tela agora são as rotas.

Arquivo: src/app/app.ts

```

import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  imports: [RouterOutlet],
  selector: 'app-root',
  styleUrls: ['./app.css'],
  templateUrl: './app.html',
})
export class App {}

```

Arquivo: src/app/app.html

```
<router-outlet />
```

5. Executando e testando a aplicação

Com todos os arquivos salvos, suba o servidor de desenvolvimento:

```
$ npm start
```

Ou, de forma equivalente:

```
$ ng serve
```

Abra o navegador em <http://localhost:4200> e teste os cenários abaixo.

Roteiro de teste

- Ao abrir a página, você é redirecionado automaticamente para `/login` (rota raiz → `redirectTo: 'login'`).
- Deixe um campo em branco e clique em Entrar → aparece "Preencha usuário e senha.", sem nenhuma requisição sendo enviada.
- Preencha com username emilys e password errada → a API responde 400 e aparece "Usuário ou senha inválidos."
- Preencha com username emilys e password emilyspass → login bem-sucedido, você é levado para `/perfil`, e a seção "Chamada autenticada a `/auth/me`" carrega o papel (role) do usuário — prova de que o interceptor anexou o token sozinho.
- Tente acessar <http://localhost:4200/perfil> direto pela URL, sem estar logado (ou depois de clicar em Sair) → o `authGuard` bloqueia e redireciona para `/login`.
- Clique em Sair → a sessão é limpa e você volta para `/login`.
- Com a sessão ativa, abra o DevTools, vá em Application > Session Storage e altere manualmente o valor salvo em `dummyjson-usuario`, trocando o `accessToken` por um texto qualquer. Recarregue `/perfil` → a chamada a `/auth/me` falha com 401, e o interceptor desloga e redireciona para `/login` automaticamente.

6. Testes automatizados

O Angular 22 usa o Vitest como executor de testes por padrão.

6.1 src/app/app.spec.ts

Como o App agora depende do Router e do HttpClient (por causa das rotas lazy e do interceptor), o teste precisa fornecer `provideRouter(routes)`, `provideHttpClient()` e `provideHttpClientTesting()` para que o TestBed consiga montar o componente sem acessar a internet de verdade.

Arquivo: src/app/app.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { provideHttpClientTesting } from '@angular/common/http/testing';

import { App } from './app';
import { routes } from './app.routes';

describe('App', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
      providers: [
        provideRouter(routes),
        provideHttpClient(),
        provideHttpClientTesting(),
      ],
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(App);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });
});
```

6.2 src/app/core/services/auth.spec.ts

Verifica se o AuthService é criado corretamente pela injeção de dependência, e se ele começa sem sessão ativa quando não há nada salvo em sessionStorage.

Arquivo: src/app/core/services/auth.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { AuthService } from './auth';
```

```
describe('AuthService', () => {
  let service: AuthService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient()],
    });
    service = TestBed.inject(AuthService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should start unauthenticated when there is no saved session', () => {
    expect(service.estaAutenticado()).toBe(false);
  });
});
```

6.3 Rodando os testes

```
$ ng test --watch=false
```

O resultado esperado é a mensagem de todos os testes passando ("Tests ... passed").

7. Build de produção

Quando o projeto estiver pronto para publicar, gere a versão otimizada com:

```
$ ng build
```

Os arquivos finais são gerados na pasta `dist/DummyJSON`, prontos para hospedar em qualquer servidor de arquivos estáticos.

Atenção: como esta aplicação guarda o token em `sessionStorage` e depende de um Route Guard no front-end, lembre-se de que a proteção real dos dados continua sendo responsabilidade da API — o Guard só evita que a interface mostre uma tela que o usuário não deveria ver.